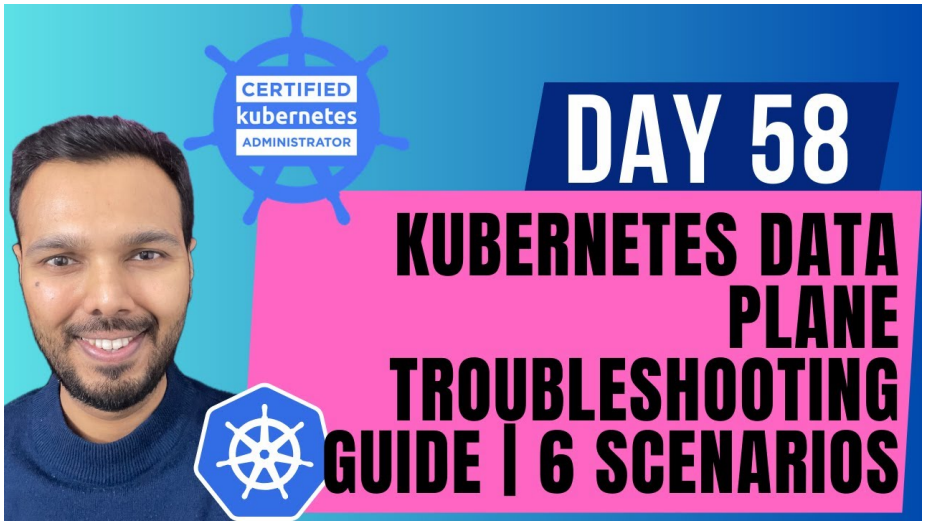
# Day 58: Kubernetes Data Plane Troubleshooting Guide | CKA Course 2025

### Video reference for Day 58 is the following:




## Table of Contents

---

## Introduction

Day 58 is your **data-plane troubleshooting sprint**. Here we focus on the components that actually run and route your workloads—**kubelet**, **container runtime (containerd/CRI-O)**, **CNI**, **kube-proxy**, **CoreDNS**, and **CSI**—and fix the highest-probability failures fast, **CKA-style**. Use the 90-second triage to size up the cluster, then work in this order: **Node/kubelet → CNI/Pod IPs → Services/Endpoints (kube-proxy) → DNS (CoreDNS) → Storage (CSI) → App (probes/limits).** Change the **minimum** required, verify immediately, and move on.

**Quick refresher: the data-plane pieces**

**kubelet (node agent)**

- Registers the node, starts/stops Pods via the CRI, runs probes, reports status.
- Reads `kubelet.conf`; talks to the API server and container runtime.
- If unhealthy → node **NotReady**, probes/exec/logs may stall.
- Think: "Is the node alive and manageable?"

**Container runtime (containerd/CRI-O)**

- Pulls images, creates containers/sandboxes, exposes the **CRI socket** to kubelet.
- If down/mispointed → kubelet can't create/attach pods; pulls fail.
- Existing containers may keep running, but are unmanaged.
- Think: "CRI up and kubelet pointing to the right socket?"

**CNI (pod networking)**

- Assigns Pod IPs, wires veth/route rules, enforces network policy (via plugin).
- Config lives in `/etc/cni/net.d`; plugin runs as a DaemonSet.
- If missing/unhealthy → Pods stuck **ContainerCreating** ("CNI plugin not initialized").
- Think: "Does a new Pod get an IP immediately?"

**kube-proxy (traffic director)**

- Programs iptables/IPVS to map **Service** VIPs/NodePorts → healthy Pod endpoints.
- Watches Endpoints/EndpointSlices; honors sessionAffinity, externalTrafficPolicy.
- If unhealthy → ClusterIP/NodePort traffic fails or is flaky.
- Think: "Do Services have endpoints, and is kube-proxy applying rules?"

**CoreDNS (cluster DNS)**

- Resolves service names for Pods via the `kube-dns` Service (ClusterIP).
- Forwards externals upstream; depends on kube-proxy and endpoints.
- If down/scaled to 0 → `nslookup kubernetes.default` fails in Pods.
- Think: "Can a Pod resolve service names right now?"

**CSI (storage interface)**

- Provisions and mounts volumes via controller/node plugins; uses **StorageClass/PVC/PV**.
- If controller/node driver is unhealthy → **PVC Pending** or mount errors.
- Defaults matter (a default SC avoids PVCs hanging).
- Think: "Do PVCs bind, and do Pods mount successfully?"

---

## Read This First: How to Use the Scenarios

These are **exam-style, high-probability** data-plane failures. Each scenario is written as **Problem → Likely causes → Checks → Fix**, with **minimal changes** and quick verification. Assume the lab stack: **kubeadm + containerd + Calico**, and stay inside the cluster (no cloud/firewall specifics).

**Approach (fast path)**

- Start at **node health**: kubelet/CRI, conditions, pressure.
- Then **CNI & Pod IPs**: `/etc/cni/net.d/`, CNI DaemonSet, pod gets an IP.
- Next **Services**: selectors → **Endpoints**; then **kube-proxy** health.
- Then **DNS**: CoreDNS pods + in-pod lookups.
- Storage next (**PVC/CSI**), then **app-level** issues (probes/limits).
- **Read Events first**; `describe` usually spells out the failure.
- Make the **smallest fix** that turns signals green; verify immediately.
- Use a throwaway **busybox** pod for `nslookup`/`wget` checks; clean up.

With that flow in mind, jump into the scenarios.

---

## 1) Node NotReady / kubelet not registered

**Problem** `kubectl get nodes` shows **NotReady** (or the node never registers); new Pods won't schedule there.

**Likely causes → How to fix**

- **kubelet service stopped / stuck** *Fix:* `sudo systemctl start kubelet` → recheck `kubectl get nodes`.
- **Swap ON** *Fix:* `sudo swapoff -a` (and remove swap from `/etc/fstab`), then `sudo systemctl start kubelet`.
- **kubelet kubeconfig/certs bad** (wrong `server:` port, bad CA path, expired cert) *Fix:* Restore **/etc/kubernetes/kubelet.conf** (correct `server: https://<apiserver>:6443`, valid CA). `sudo systemctl restart kubelet`.
- **Container runtime down (containerd/cri-o)** *Fix:* `sudo systemctl start containerd` (or cri-o), then restart kubelet.
- **Clock skew / DNS to API** *Fix:* `sudo systemctl restart systemd-timesyncd` (or chrony); fix `/etc/hosts`/DNS; ensure `:6443` reachable.
- **Resource pressure (Disk/Memory/PIDs)** *Fix:* Free disk/inodes, lower load; kubelet comes back once thresholds clear.
- *(Optional)* **cgroup driver mismatch** *Fix:* Align kubelet & runtime drivers; restart both.

**Quick checks (with what they tell you)**

```
# Cluster view
kubectl get nodes -o wide
kubectl describe node <node>          # Conditions: Ready, Memory/Disk/PID pressure

# On the node
sudo systemctl status kubelet         # service up?
sudo journalctl -u kubelet -b -o cat | tail -n 80
sudo systemctl status containerd      # runtime up?
crictl info | head -20                # CRI & cgroup driver

# Swap / resources
swapon --show
free -h
df -h

# API reachability & time
getent hosts <api-server-hostname>
```

```
nc -vz <api-server-hostname> 6443    # or: curl -vk https://<api>:6443
timedatectl
```

**Symptom clues in logs (what to look for)**

- **Swap:** `running with swap on is not supported`
- **Certs/CA:** `x509: certificate signed by unknown authority`, `no such file or directory` (CA/key), `certificate has expired or is not yet valid`
- **API connect:** `failed to run Kubelet: cannot connect ... :6443`, `connection refused`, `i/o timeout`
- **Runtime/CRI:** `container runtime network not ready`, `failed to connect to CRI`
- **Pressure:** `eviction manager: ...` / `imagefs has insufficient space`

---

## 2) Pods stuck ContainerCreating / CNI not initialized

**Problem** Pods never get IPs; Events show *CNI plugin not initialized* or *failed to set up sandbox*.

**Likely causes → How to fix**

- **CNI DaemonSet not running / scaled to 0** *Fix:* `kubectl -n kube-system get ds` → scale DS back or re-apply CNI manifest.
- **`/etc/cni/net.d` missing/empty or wrong plugin conf** *Fix:* Restore valid CNI config (or re-install add-on), then `sudo systemctl restart kubelet`.
- **Wrong Pod CIDR / mismatched cluster networking config** *Fix:* Align CNI config with cluster Pod CIDR; re-apply and restart kubelet.

**Checks (quick and surgical)**

```
kubectl -n kube-system get pods -o wide | egrep -i 'calico|cilium|flannel|weave'     # CNI p
kubectl -n kube-system get ds -o wide | egrep -i 'calico|cilium|flannel|weave'       # DS de
kubectl describe pod <p> | sed -n '/Events:/,$p'                                     # exac
ls -l /etc/cni/net.d/                                                               # conf
sudo journalctl -u kubelet -b -o cat | egrep -i 'cni|sandbox|plugin|network' | tail  # kubel
```

**Fix (minimal moves, then verify)**

```
# If DS was scaled down or crashed:
kubectl -n kube-system rollout restart ds/<cni-daemonset>   # or scale back to desired

# If config missing:
```

```
# (restore your backup or re-apply the CNI add-on manifest)
sudo systemctl restart kubelet

# Verify:
kubectl delete pod <p>; kubectl get pod <p> -o wide          # pod gets IP, goes Running
```

**Example** `Events: CNI plugin not initialized` + empty `/etc/cni/net.d` → re-apply CNI add-on → restart kubelet → new Pod gets IP and **Running**.

---

## 3) Pod-to-Pod across nodes / Service ClusterIP not reachable

**Problem** Pods on different nodes can't talk; `curl http://<svc-name>:<port>` (or `curl <ClusterIP>:<port>`) fails.

### Likely causes → How to fix

- **Service has no endpoints** (selector mismatch / pods not Ready) *Fix:* Correct the Service **selector** to match pod labels; ensure backends are **Running & Ready**.
- **`targetPort` wrong / app not listening** *Fix:* Set Service **`targetPort`** to the container's actual listening port (or update the container to listen on the declared port).
- **kube-proxy not programming rules** (DS unhealthy) *Fix:* Check kube-proxy DaemonSet; restart/rollout if needed.
- **NetworkPolicy blocking traffic** *Fix:* Inspect policies in the namespace; add an allow rule or remove the blocking policy.

### Checks (quick)

```
# Service & backends
kubectl get svc <svc> -o wide
kubectl describe svc <svc>                  # selector, port/targetPort
kubectl get endpoints,endpointslices <svc> -o wide
kubectl get pods -l <svc-selector> -o wide  # Ready=TRUE? correct labels?

# kube-proxy health
kubectl -n kube-system get ds kube-proxy -o wide
kubectl -n kube-system logs -l k8s-app=kube-proxy --tail=100

# NetworkPolicy (namespace-wide)
kubectl get netpol
```

**Fix (minimal moves, then verify)**

```
# If endpoints are empty: fix selector or pod labels
kubectl patch svc <svc> -p '{"spec":{"selector":{"app":"<correct-label>"}}}'

# If targetPort is wrong: set it to the container's port
kubectl patch svc <svc> -p '{"spec":{"ports":[{"port":80,"targetPort":8080}]}}'  # example

# If kube-proxy unhealthy:
kubectl -n kube-system rollout restart ds/kube-proxy

# If NetworkPolicy blocks traffic: adjust or remove the policy
# (e.g., temporarily allow all within the namespace for verification)

# Verify:
kubectl get endpoints <svc> -o wide
kubectl run curl --image=busybox:1.36 --restart=Never -- sh -c 'wget -qO- http://<svc>:<port
```

**Example** Endpoints show `<none>` → fix Service selector to match pod label (e.g., `app=web`) → endpoints populate → `wget http://<svc>:<port>` succeeds.

---

## 4) DNS failures (CoreDNS)

**Problem** Pods say **no such host**; Service names don't resolve.

**Likely causes**

- CoreDNS pods down/crashing
- Kubelet `--cluster-dns` wrong; pod `/etc/resolv.conf` points to bad IP
- NetworkPolicy blocks DNS; CoreDNS can't reach upstream

**Checks**

```
kubectl -n kube-system get pods -l k8s-app=kube-dns -o wide    # CoreDNS healthy?
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=80      # loop/forward errors?
# From a test pod:
nslookup kubernetes.default.svc.cluster.local 10.96.0.10       # replace with your cluster I
cat /etc/resolv.conf                                           # search/domain/options insid
kubectl get svc -n kube-system kube-dns -o wide                # ClusterIP of DNS
```

**Fix**

- Restart/fix CoreDNS; correct the cluster DNS IP used by kubelet.
- Remove blocking NetworkPolicies; ensure upstream resolvers are reachable.

**Example** Pod `resolv.conf` shows a wrong nameserver → set kubelet `--cluster-dns=10.96.0.10`, restart kubelet; name lookups work.

---

## 5) ImagePullBackOff / ErrImagePull

**Problem** Pod won't start because the image can't be pulled.

**Likely causes → How to fix**

- **Wrong image name/tag** *Fix:* Point `image:` to a valid repo:tag; redeploy.
- **Private registry requires auth** *Fix:* Create an `imagePullSecret` and reference it in the Pod/ServiceAccount.
- **Stale local image / policy** *Fix:* Ensure correct tag; delete the Pod (or `rollout restart`) to force a fresh pull.

**Checks (quick)**

```
kubectl describe pod <p> | sed -n '/Events:/,$p'   # shows the exact pull error
crictl pull <image:tag>                            # try pulling from the node
kubectl get sa <sa> -o yaml | grep -A2 imagePullSecrets  # is the secret attached?
```

**Fix (minimal, then verify)**

```
# Example for private registry:
kubectl create secret docker-registry regcred \
  --docker-server=<REG> --docker-username=<U> --docker-password=<P> --docker-email=<E>
# Reference in Pod spec or attach to the ServiceAccount
kubectl delete pod <p>   # force re-pull
kubectl get pods -w
```

**Example** Events show `manifest unknown` → fix **tag** → Pod pulls and becomes **Running**.

---

## 6) CrashLoopBackOff / OOMKilled / probe failures

**Problem** Container starts then repeatedly exits; readiness never becomes **Ready**.

**Likely causes → How to fix**

- **Bad command/args or missing config/volume** *Fix:* Correct `command/args`, env, mounts; reapply.
- **Probes too aggressive** (app needs warm-up) *Fix:* Relax `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`.
- **OOMKilled** (exit 137) / limits too low *Fix:* Raise memory **requests/limits** (or reduce app usage).

### Checks (quick)

```
kubectl logs <p> --previous
kubectl describe pod <p> | sed -n '/State:/,/Events:/p'   # OOMKilled? ExitCode?
kubectl get deploy <d> -o yaml | sed -n '/livenessProbe:/,/readinessProbe:/p'
```

### Fix (minimal, then verify)

```
# Example: bump memory & restart
kubectl set resources deploy/<d> --requests=memory=256Mi --limits=memory=512Mi
kubectl rollout restart deploy/<d>
kubectl get pods -w
```

**Example** Describe shows **OOMKilled** → increase memory limits → Pod stabilizes.

---

## 7) PVC Pending / volume mount errors (CSI)

**Problem** Pod stuck **Pending** (PVC not Bound) or **CreateContainerError** (mount/permission).

### Likely causes → How to fix

- **No default StorageClass / wrong accessModes/size** *Fix:* Mark an SC as **default**; align PVC size & access modes.
- **CSI driver unhealthy** *Fix:* Ensure CSI controller/node Pods are **Running**; then retry.
- **Bad mount spec (name/path/fsType/permissions)** *Fix:* Correct `volumeMounts`/`volumes` names & paths; set proper `fsType` or `fsGroup` if needed.

### Checks (quick)

```
kubectl get pvc,pv,sc
kubectl describe pvc <pvc>                    # why not Bound?
kubectl -n kube-system get pods | grep -i csi # driver health
kubectl describe pod <p> | sed -n '/Mounts:/,/Events:/p'
```

### Fix (minimal, then verify)

```
# Make an SC default (example)
kubectl annotate sc <sc-name> storageclass.kubernetes.io/is-default-class=true --overwrite
# Retry the workload
kubectl delete pod <p> && kubectl apply -f <pod>.yaml
kubectl get pvc && kubectl get pod <p> -o wide
```

**Example** No default SC → PVC **Pending** → mark SC default → PVC **Bound** → Pod **Running**.

---

## 8) NodePort / LoadBalancer not reachable

**Problem** `curl http://<nodeIP>:<nodePort>` (or via LB) fails.

### Likely causes → How to fix

- **Service selector/targetPort mismatch** *Fix:* Match Service **selector** to pod labels; set correct **targetPort**.
- **`externalTrafficPolicy: Local` with no local endpoints** *Fix:* Schedule a backend on that node or set policy to **Cluster**.
- **kube-proxy rules not present** (DS unhealthy) *Fix:* Check kube-proxy DaemonSet; restart/rollout.

### Checks (quick)

```
kubectl describe svc <svc>                    # type, nodePort, targetPort, policy
kubectl get endpoints <svc> -o wide           # endpoints exist? on that node for Local?
kubectl -n kube-system get ds kube-proxy -o wide
# On the node, is kube-proxy listening on the nodePort?
sudo ss -lnt 'sport = :<nodePort>'
```

### Fix (minimal, then verify)

```
# Fix selector or targetPort as needed (patch or reapply manifest)
# If policy=Local, ensure a backend Pod on that node
kubectl -n kube-system rollout restart ds/kube-proxy   # if kube-proxy unhealthy

kubectl get endpoints <svc> -o wide
```

**Example** Service set to **Local** but node has **no endpoint** → place a Pod on that node (or switch to **Cluster**) → NodePort works.

---

### Conclusion

Data-plane issues are rarely mysterious when you follow a disciplined path. Verify the node agent (**kubelet + CRI**) first, confirm Pods are getting **IPs** (CNI), ensure Services have **endpoints** and **kube-proxy** is programming rules, validate **DNS** from inside a Pod, then check **PVC/CSI** and finally the **application** (probes/resources/args). Read **Events** before editing anything, apply the smallest fix that turns signals green, and always re-test with a quick busybox Pod. That's the CKA way: **fast, minimal, verified**.

---

**References**


- Kubernetes Docs — **Troubleshooting Clusters & Nodes** https://kubernetes.io/docs/tasks/debug/
- Kubernetes Docs — **Nodes & Kubelet** (config, certificates, logs) https://kubernetes.io/docs/concepts/architecture/nodes/ https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/
- Kubernetes Docs — **Container Runtime Interface (CRI)** https://kubernetes.io/docs/setup/production-environment/container-runtimes/
- Kubernetes Docs — **Cluster Networking & Services** https://kubernetes.io/docs/concepts/cluster-administration/networking/ https://kubernetes.io/docs/concepts/services-networking/service/
- Kubernetes Docs — **kube-proxy** https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/
- Kubernetes Docs — **DNS for Services and Pods** (CoreDNS) https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/
- CoreDNS Docs — **Kubernetes plugin** & ops notes https://coredns.io/plugins/kubernetes/
- Kubernetes Docs — **Storage: PV/PVC/StorageClass** & **CSI** https://kubernetes.io/docs/concepts/storage/ https://kubernetes.io/docs/concepts/storage/volumes/ https://kubernetes.io/docs/concepts/storage/storage-classes/
- containerd — **Getting Started & Troubleshooting** https://github.com/containerd/containerd/blob/main/docs/
- Calico (if you're using it) — **Install & Troubleshoot** https://docs.tigera.io/calico/latest/


---